WEB SECURITY LAB

Tshepang Suping

Table of Contents

Overview 2
Clone website 3
Database setup SQLite 12
  Downloading SQLite 12
  Creating sqlite3 table 14
Linking webpage to database 15
Testing the Login webpage 18
SQL injection 23
Cross- Site Scripting 26
References 28

Overview

I was tasked with creating an offline clone of AltoroMutual bank login page whereby two major web vulnerabilities, SQL Injection (SQLi) and Cross-Site Scripting (XSS) explored. An SQLite database was used to store client information and also serve as a reminder to how malicious inputs can reveal personal data.

SQL Injection (SQLi)

SQL Injection is a security vulnerability in web applications that attackers exploit by inserting malicious SQL code through user input. This gains them access to sensitive data, most times take control over the system and change database contents (geeksforgeeks, 2025).

The types of SQL Injections:

- Error-Based SQL Injection
- Union-Based SQL Injection
- Blind-Based SQL Injection

Cross-Site Scripting (XSS)

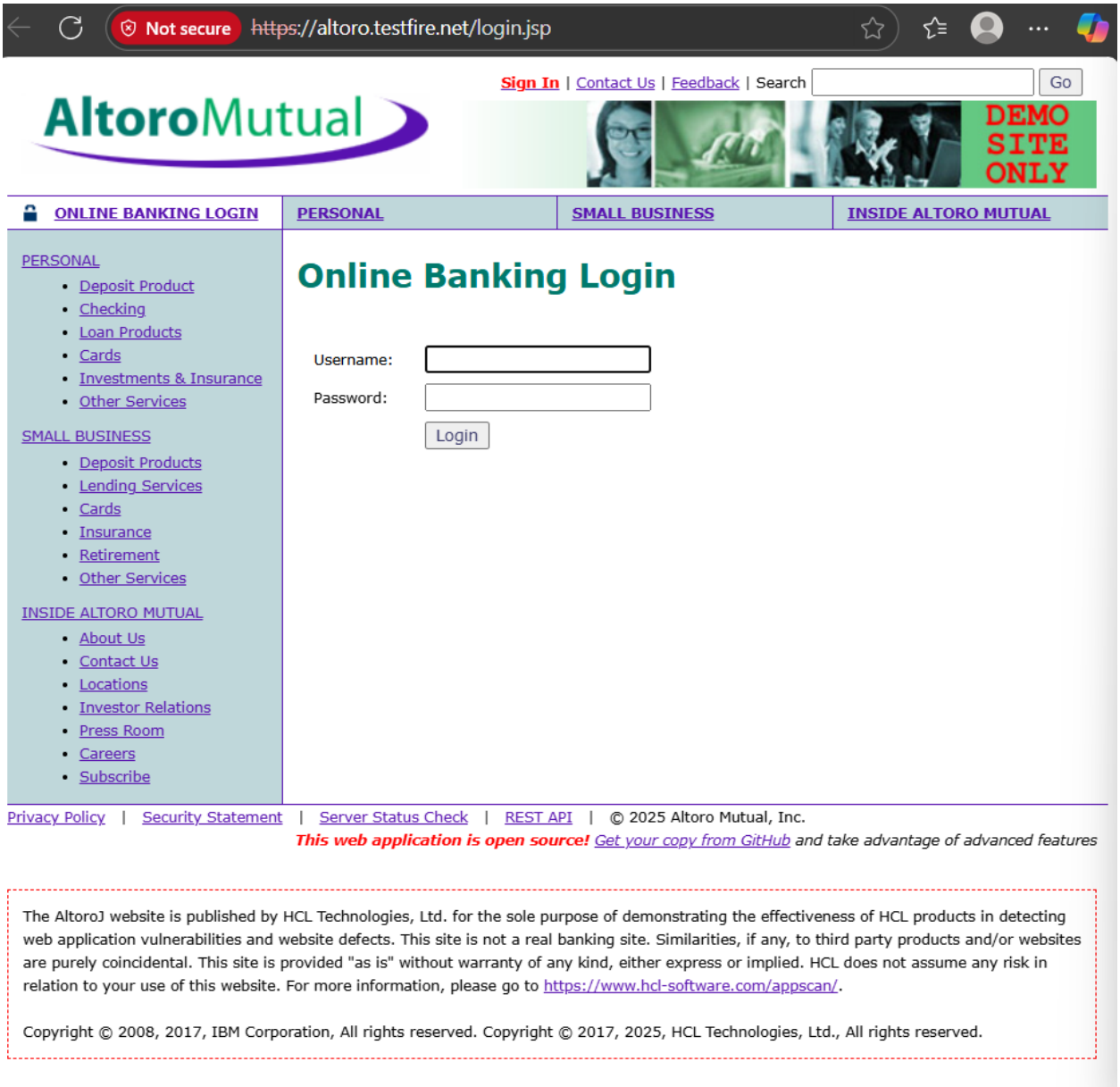
Cross-Site Scripting are a type of injection attack where malicious scripts are injected into trusted websites. Attacks take place when a web application is used to send malicious code usually in the form of a browser script to a target user. Flaws are what fuels these attacks into succeeding and happen when a web application makes use of a user within the output then generates without validating it (Jim Manico, n.d.).

The types of XSS Attacks are:

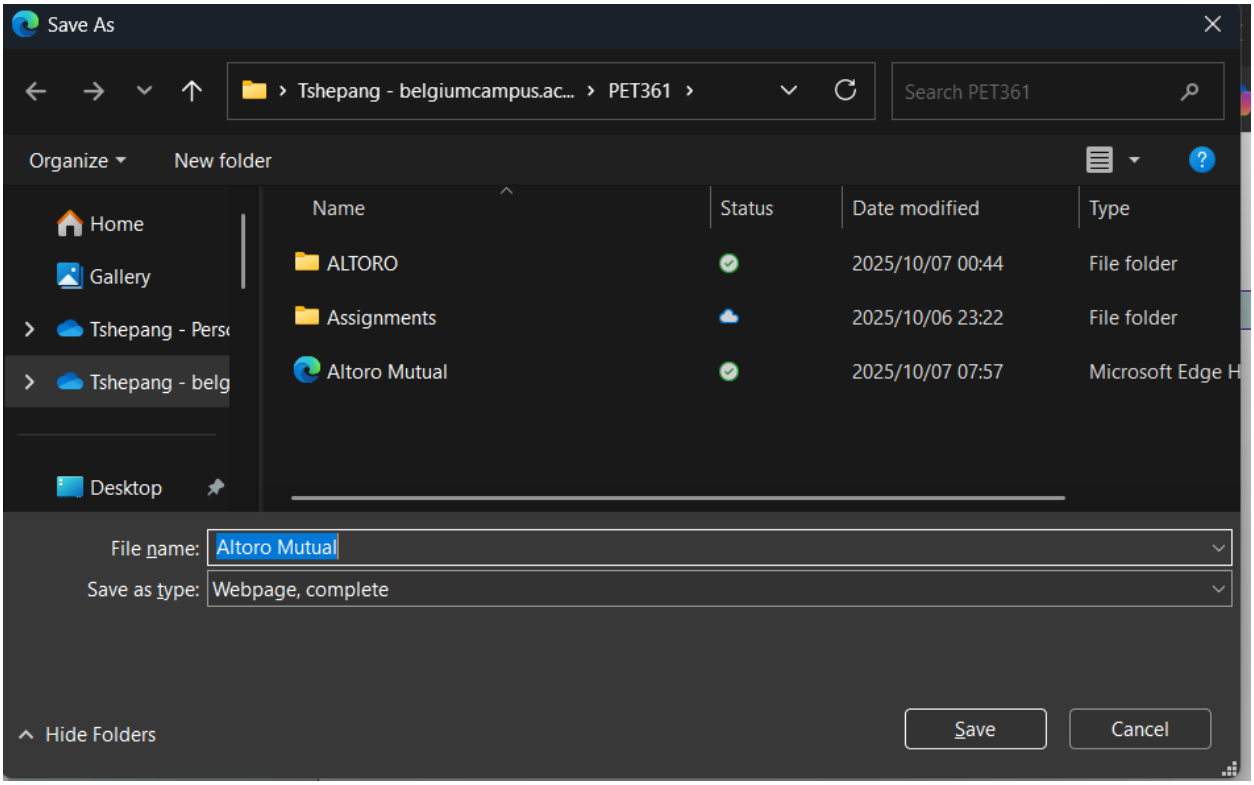
- Reflected XSS Attacks
- Stored XSS Attacks
- Blind Cross-site Scripting

In this milestone, Blind-Based SQL Injection and Reflected XSS Attacks was used to successfully complete the given task.

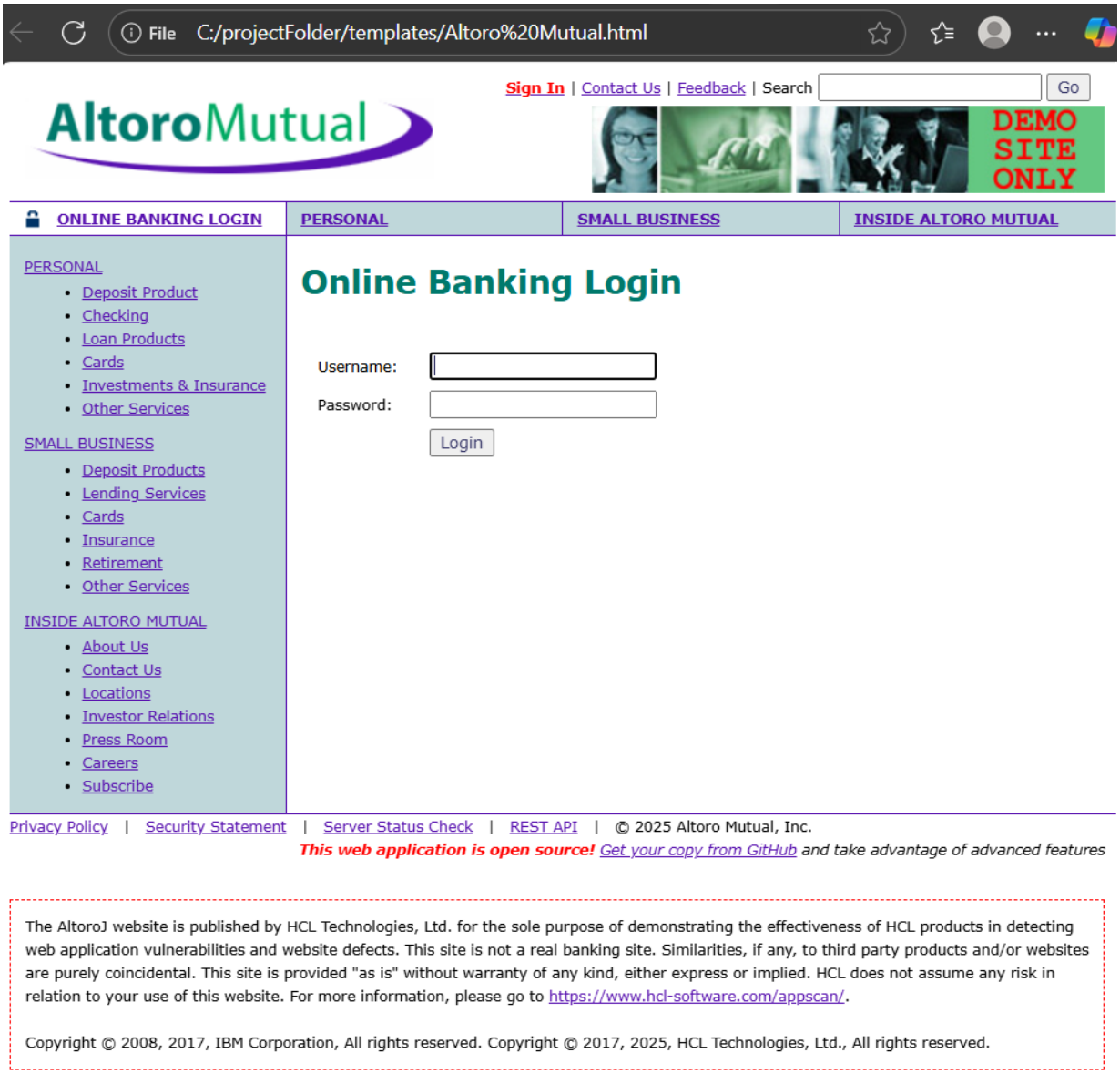
Clone website



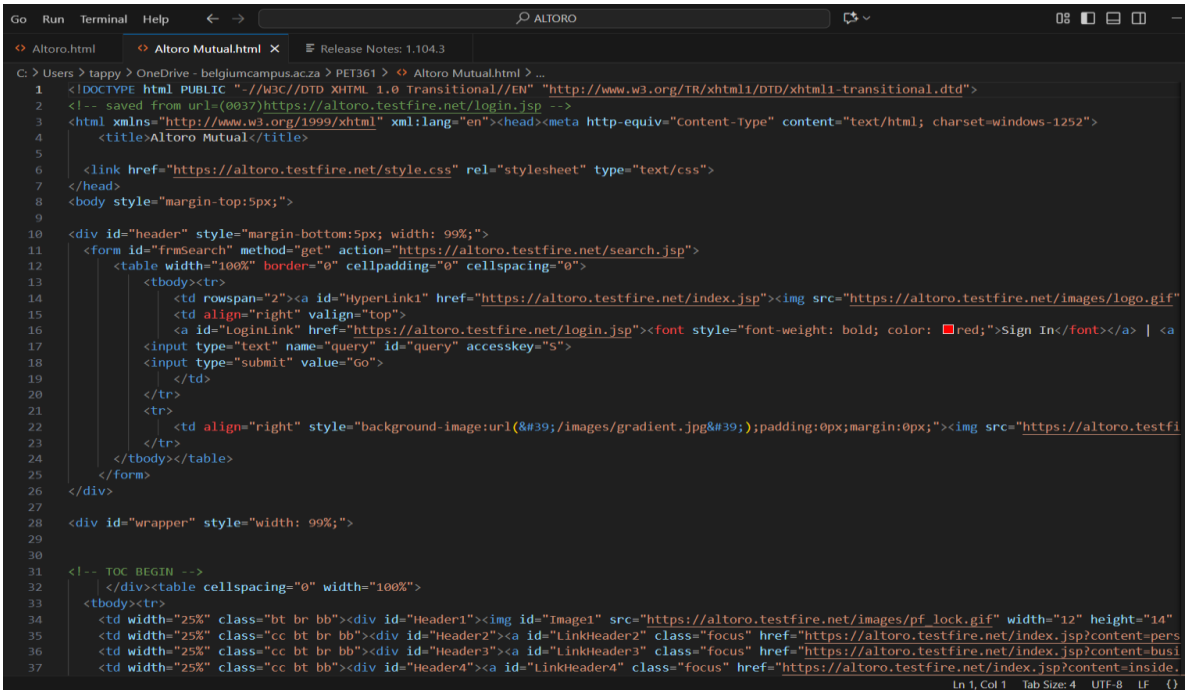
This is the original Altoro Mutual demo site only webpage. Which can be found here: Altoro Mutual.



Using **ctrl S** to save the Altoro Mutual webpage in the desired folder and then viewing the code on virtual studio code (OHkae, 2021).



The cloned Altoro webpage saved at desired folder before configuration. The path the cloned Altoro webpage is saved in, **C;/projectFolder/templates/Altoro%20Mutual.html** .



This is the cloned code successfully saved before editing. The actual code is pasted here:


```
<!-- BEGIN HEADER -->

<!DOCTYPE html PUBLIC "-//W3C//DTD XHTML 1.0 Transitional//EN" "http://www.w3.org/TR/xhtml1/DTD/xhtml1-transitional.dtd">

<html xmlns="http://www.w3.org/1999/xhtml" xml:lang="en" >


<head>

<title>Altoro Mutual</title>

<meta http-equiv="Content-Type" content="text/html; charset=iso-8859-1" />

<link href="/style.css" rel="stylesheet" type="text/css" />

</head>

<body style="margin-top:5px;">

<div id="header" style="margin-bottom:5px; width: 99%;">

<form id="frmSearch" method="get" action="/search.jsp">

<table width="100%" border="0" cellpadding="0" cellspacing="0">

<tr>

<td rowspan="2"><a id="HyperLink1" href="/index.jsp"><img src="/images/logo.gif" width=283 height=80/></a></td>
```

<td align="right" valign="top">

<a id="LoginLink" href="/login.jsp"><font style="font-weight: bold; color: red;">Sign In</font></a> | <a id="HyperLink3" href="/index.jsp?content=inside_contact.htm">Contact Us</a> | <a id="HyperLink4" href="/feedback.jsp">Feedback</a> | <label for="txtSearch">Search</label>

<input type="text" name="query" id="query" accesskey="S" />

<input type="submit" value="Go" />

</td>

</tr>

<tr>

<td align="right" style="background-image:url('/images/gradient.jpg');padding:0px;margin:0px;"><img src="/images/header_pic.jpg" alt="" width=354 height=60/></td>

</tr>

</table>

</form>

<table cellspacing="0" width="100%">

<tr>

<td width="25%" class="bt br bb"> <img id="Image1" src="/images/pf_lock.gif" width=12 height=14 style="vertical-align: bottom;" alt="Secure Login"/>   <a id="AccountLink" href="/login.jsp" class="focus" >ONLINE BANKING LOGIN</a> </td>

<td width="25%" class="cc bt br bb"> <a id="LinkHeader2" class="focus" href="/index.jsp?content=personal.htm" >PERSONAL</a> </td>

<td width="25%" class="cc bt br bb"> <a id="LinkHeader3" class="focus" href="/index.jsp?content=business.htm" >SMALL BUSINESS</a> </td>

<td width="25%" class="cc bt bb"> <a id="LinkHeader4" class="focus" href="/index.jsp?content=inside.htm">INSIDE ALTORO MUTUAL</a> </td>

</tr>

<tr>

<!-- END HEADER -->

<!-- TOC BEGIN -->

<td valign="top" class="cc br bb">

<br style="line-height: 10px;"/>

```
<a id="CatLink1" class="subheader" href="index.jsp?content=personal.htm">PERSONAL</a>

<ul class="sidebar">

<li><a id="MenuHyperLink1" href="index.jsp?content=personal_deposit.htm">Deposit Product</a></li>

<li><a id="MenuHyperLink2" href="index.jsp?content=personal_checking.htm">Checking</a></li>

<li><a id="MenuHyperLink3" href="index.jsp?content=personal_loans.htm">Loan Products</a></li>

<li><a id="MenuHyperLink4" href="index.jsp?content=personal_cards.htm">Cards</a></li>

<li><a id="MenuHyperLink5" href="index.jsp?content=personal_investments.htm">Investments &amp; Insurance</a></li>

<li><a id="MenuHyperLink6" href="index.jsp?content=personal_other.htm">Other Services</a></li>

</ul>

<a id="CatLink2" class="subheader" href="index.jsp?content=business.htm">SMALL BUSINESS</a>

<ul class="sidebar">

<li><a id="MenuHyperLink7" href="index.jsp?content=business_deposit.htm">Deposit Products</a></li>

<li><a id="MenuHyperLink8" href="index.jsp?content=business_lending.htm">Lending Services</a></li>

<li><a id="MenuHyperLink9" href="index.jsp?content=business_cards.htm">Cards</a></li>

<li><a id="MenuHyperLink10" href="index.jsp?content=business_insurance.htm">Insurance</a></li>

<li><a id="MenuHyperLink11" href="index.jsp?content=business_retirement.htm">Retirement</a></li>

<li><a id="MenuHyperLink12" href="index.jsp?content=business_other.htm">Other Services</a></li>

</ul>

<a id="CatLink3" class="subheader" href="index.jsp?content=inside.htm">INSIDE ALTORO MUTUAL</a>

<ul class="sidebar">

<li><a id="MenuHyperLink13" href="index.jsp?content=inside_about.htm">About Us</a></li>

<li><a id="MenuHyperLink14" href="index.jsp?content=inside_contact.htm">Contact Us</a></li>

<li><a id="MenuHyperLink15" href="cgi.exe">Locations</a></li>

<li><a id="MenuHyperLink16" href="index.jsp?content=inside_investor.htm">Investor Relations</a></li>

<li><a id="MenuHyperLink17" href="index.jsp?content=inside_press.htm">Press Room</a></li>

<li><a id="MenuHyperLink18" href="index.jsp?content=inside_careers.htm">Careers</a></li>

<li><a id="MenuHyperLink19" href="subscribe.jsp">Subscribe</a></li>

</ul>

</td>
```

```
<!-- TOC END -->

<td valign="top" colspan="3" class="bb">

<div class="fl" style="width: 99%;">

<h1>Online Banking Login</h1>

<!-- To get the latest admin login, please contact SiteOps at 415-555-6159 -->

<p><span id="_ctl0__ctl0_Content_Main_message" style="color:#FF0066;font-size:12pt;font-weight:bold;">

</span></p>

<form action="doLogin" method="post" name="login" id="login" onsubmit="return (confirminput(login));">

<table>

<tr>

<td>

Username:

</td>

<td>

<input type="text" id="uid" name="uid" value="" style="width: 150px;">

</td>

<td>

</td>

</tr>

<tr>

<td>

Password:

</td>

<td>

<input type="password" id="passw" name="passw" style="width: 150px;">

</td>

</tr>

<tr>

<td></td>
```

```
<td>
<input type="submit" name="btnSubmit" value="Login">
</td>
</tr>
</table>
</form>
</div>
<script type="text/javascript">
function setfocus() {
if (document.login.uid.value==="") {
document.login.uid.focus();
} else {
document.login.passw.focus();
}
}
function confirminput(myform) {
if (myform.uid.value.length && myform.passw.value.length) {
return (true);
} else if (!(myform.uid.value.length)) {
myform.reset();
myform.uid.focus();
alert ("You must enter a valid username");
return (false);
} else {
myform.passw.focus();
alert ("You must enter a valid password");
return (false);
}
}
```

window.onload = setfocus;

</script>

</td>

<!-- BEGIN FOOTER -->

</tr>

</table>

<a id="HyperLink5" href="/index.jsp?content=privacy.htm">Privacy Policy</a>

  |

<a id="HyperLink6" href="/index.jsp?content=security.htm">Security Statement</a>

  |

<a id="HyperLink6" href="/status_check.jsp">Server Status Check</a>

  |

<a id="HyperLink6" href="/swagger/index.html">REST API</a>

  |

© 2025 Altoro Mutual, Inc.

This web application is open source! <a href="https://github.com/AppSecDev/AltoroJ/">Get your copy from GitHub</a> and take advantage of advanced features

The AltoroJ website is published by HCL Technologies, Ltd. for the sole purpose of

demonstrating the effectiveness of HCL products in detecting web application

vulnerabilities and website defects. This site is not a real banking site. Similarities,

if any, to third party products and/or websites are purely coincidental. This site is

provided "as is" without warranty of any kind, either express or implied. HCL does

not assume any risk in relation to your use of this website. For more information,

please go to <a id="HyperLink7" href="https://www.hcl-software.com/appscan/" >https://www.hcl-software.com/appscan/</a>.

Copyright © 2008, 2017, IBM Corporation, All rights reserved.

Copyright © 2017, 2025, HCL Technologies, Ltd., All rights reserved.

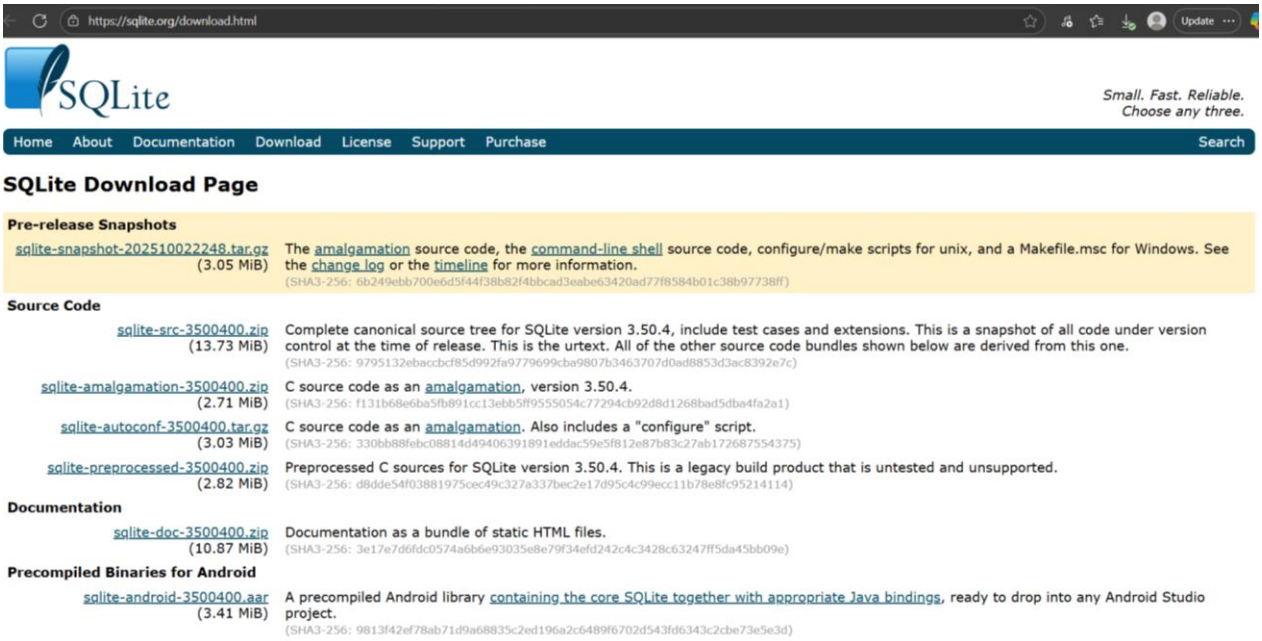
</body>

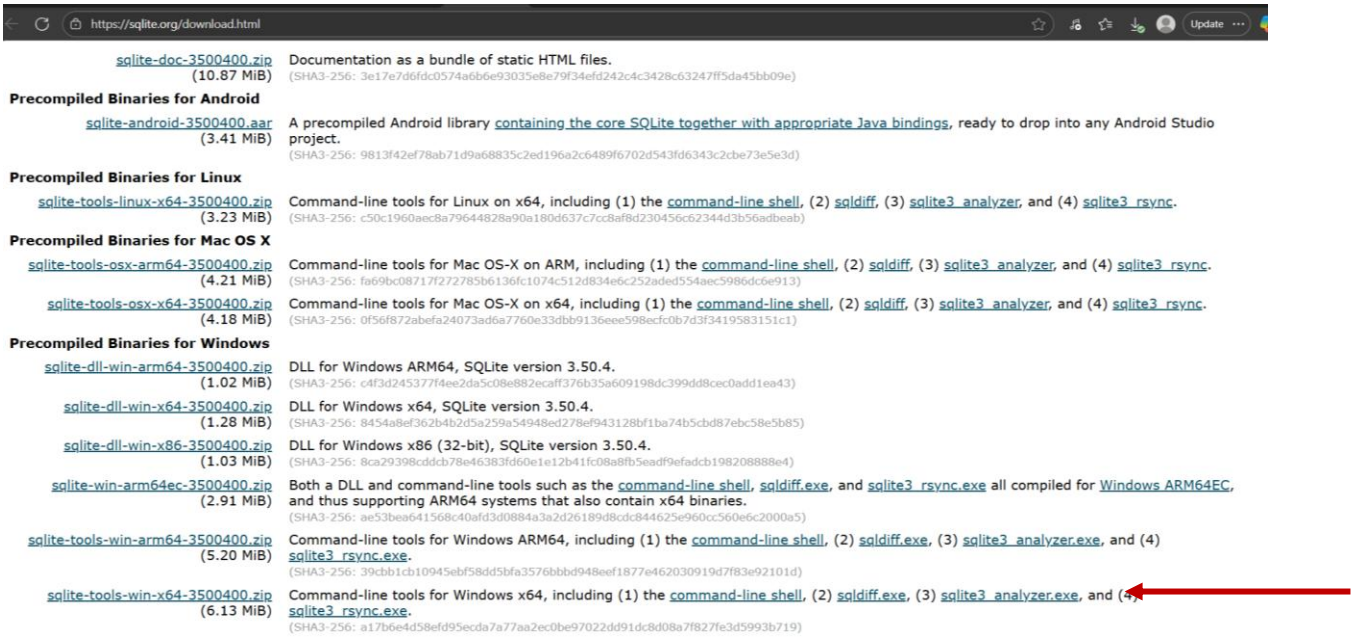
</html>

Database setup SQLite

Downloading SQLite

Begin by downloading SQLite from this website: [SQLite Download Page]

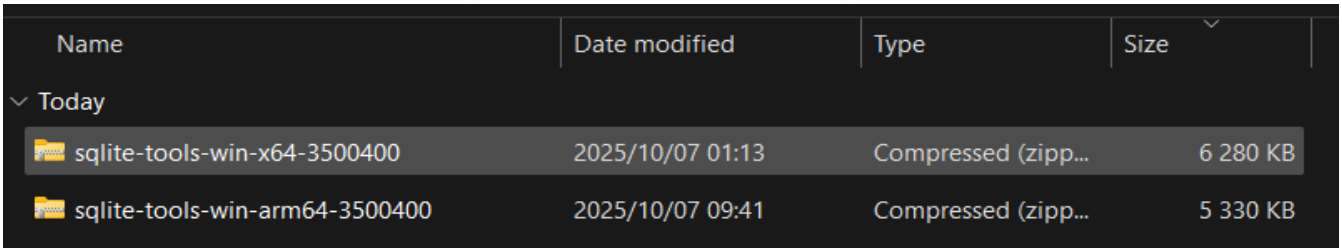




https://sqlite.org/download.html

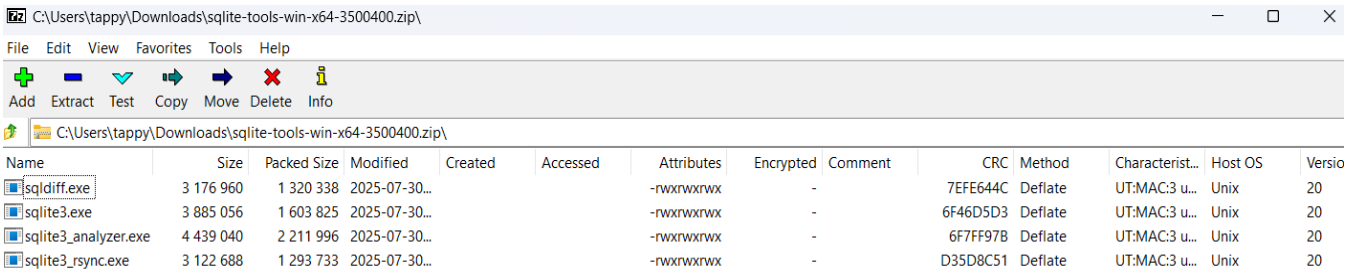
sqlite-doc-3500400.zip (10.87 MiB)	Documentation as a bundle of static HTML files. (SHA3-256: 3e17e7d6fdc0574a6b6e93035e8e79f34efd242c4c3428c63247ff5da45bb09e)
Precompiled Binaries for Android	
sqlite-android-3500400.aar (3.41 MiB)	A precompiled Android library containing the core SQLite together with appropriate Java bindings, ready to drop into any Android Studio project. (SHA3-256: 9813f42ef78ab71d9a68835c2ed196a2c6489f6702d543fd6343c2cbe73e5e3d)
Precompiled Binaries for Linux	
sqlite-tools-linux-x64-3500400.zip (3.23 MiB)	Command-line tools for Linux on x64, including (1) the command-line shell, (2) sqldiff, (3) sqlite3_analyzer, and (4) sqlite3_rsync. (SHA3-256: c50c1960aec8a79644828a90a180d637c7cc8af8d230456c62344d3b56adbeab)
Precompiled Binaries for Mac OS X	
sqlite-tools-osx-arm64-3500400.zip (4.21 MiB)	Command-line tools for Mac OS-X on ARM, including (1) the command-line shell, (2) sqldiff, (3) sqlite3_analyzer, and (4) sqlite3_rsync. (SHA3-256: fa69bc08717f272785b6136fc1074c512d834e6c252aded554aec5986dc6e913)
sqlite-tools-osx-x64-3500400.zip (4.18 MiB)	Command-line tools for Mac OS-X on x64, including (1) the command-line shell, (2) sqldiff, (3) sqlite3_analyzer, and (4) sqlite3_rsync. (SHA3-256: 0f56f872abefa24073ad6a7760e33dbb9136eee598ecfc0b7d3f3419583151c1)
Precompiled Binaries for Windows	
sqlite-dll-win-arm64-3500400.zip (1.02 MiB)	DLL for Windows ARM64, SQLite version 3.50.4. (SHA3-256: c4f3d245377f4ee2da5c08e882ecaff376b35a609198dc399dd8cec0add1ea43)
sqlite-dll-win-x64-3500400.zip (1.28 MiB)	DLL for Windows x64, SQLite version 3.50.4. (SHA3-256: 8454a8ef362b4b2d5a259a54948ed278ef943128bf1ba74b5cbd87ebc58e5b85)
sqlite-dll-win-x86-3500400.zip (1.03 MiB)	DLL for Windows x86 (32-bit), SQLite version 3.50.4. (SHA3-256: 8ca29398cddcb78e46383fd60e1e12b41fc08a8fb5eadf9efadcb198208888e4)
sqlite-win-arm64ec-3500400.zip (2.91 MiB)	Both a DLL and command-line tools such as the command-line shell, sqldiff.exe, and sqlite3_rsync.exe all compiled for Windows ARM64EC, and thus supporting ARM64 systems that also contain x64 binaries. (SHA3-256: ae53bea641568c40afd3d0884a3a2d26189d8cdc844625e960cc560e6c2000a5)
sqlite-tools-win-arm64-3500400.zip (5.20 MiB)	Command-line tools for Windows ARM64, including (1) the command-line shell, (2) sqldiff.exe, (3) sqlite3_analyzer.exe, and (4) sqlite3_rsync.exe. (SHA3-256: 39cbb1cb10945ebf58dd5bfa3576bbbd948eef1877e462030919d7f83e92101d)
sqlite-tools-win-x64-3500400.zip (6.13 MiB)	Command-line tools for Windows x64, including (1) the command-line shell, (2) sqldiff.exe, (3) sqlite3_analyzer.exe, and (4) sqlite3_rsync.exe. (SHA3-256: a17b6e4d58efd95ecda7a77aa2ec0be97022dd91dc8d08a7f827fe3d5993b719)

Under Precompiled Binaries for Windows select sqlite -tools-win-x64-3500400.zip , that is the file that should be downloaded according to (McKenzie, 2024).



Name	Date modified	Type	Size
Today			
sqlite-tools-win-x64-3500400	2025/10/07 01:13	Compressed (zipp...	6 280 KB
sqlite-tools-win-arm64-3500400	2025/10/07 09:41	Compressed (zipp...	5 330 KB

Select sqlite-tools-win-x64-3500400.zip, that is the file that will be extracted.



C:\Users\tappy\Downloads\sqlite-tools-win-x64-3500400.zip\

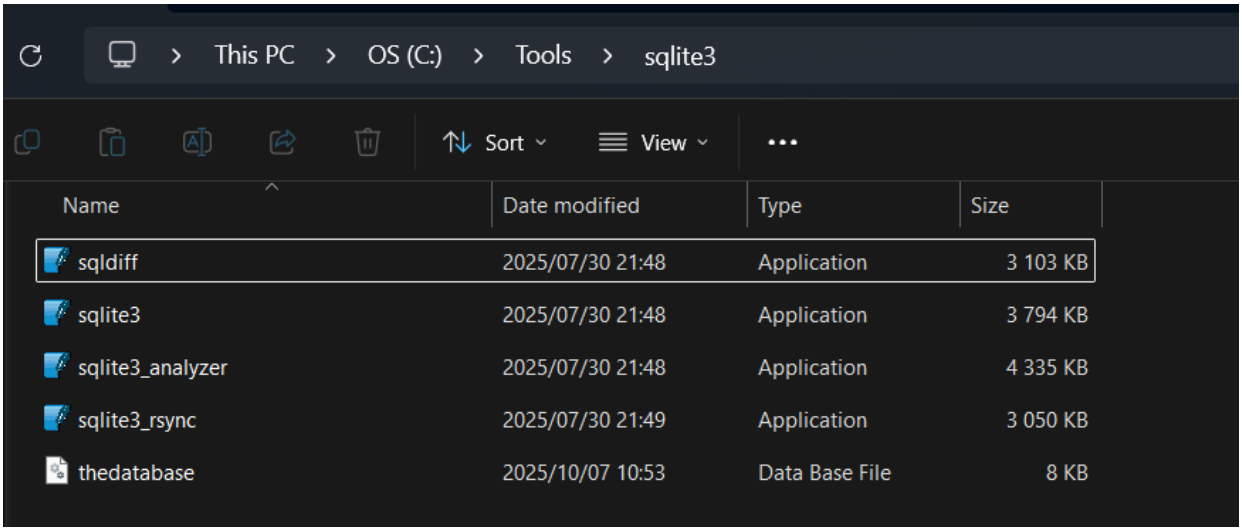
File Edit View Favorites Tools Help

Add Extract Test Copy Move Delete Info

C:\Users\tappy\Downloads\sqlite-tools-win-x64-3500400.zip\

Name	Size	Packed Size	Modified	Created	Accessed	Attributes	Encrypted	Comment	CRC	Method	Characterist...	Host OS	Versio
sqldiff.exe	3 176 960	1 320 338	2025-07-30...			-rwxrwxrwx	-		7EFE644C	Deflate	UT:MAC:3 u...	Unix	20
sqlite3.exe	3 885 056	1 603 825	2025-07-30...			-rwxrwxrwx	-		6F46D5D3	Deflate	UT:MAC:3 u...	Unix	20
sqlite3_analyzer.exe	4 439 040	2 211 996	2025-07-30...			-rwxrwxrwx	-		6F7FF97B	Deflate	UT:MAC:3 u...	Unix	20
sqlite3_rsync.exe	3 122 688	1 293 733	2025-07-30...			-rwxrwxrwx	-		D35D8C51	Deflate	UT:MAC:3 u...	Unix	20

7 zipped was used to extract the sqlite tools zipped file.



The zipped file was successfully unzipped the next step is to open the PowerShell terminal in this folder to get started on creating the database table that consists of 5 clients (McKenzie, 2024).

Creating sqlite3 table

```
PS C:\Tools\sqlite3> sqlite3 thedatabase.db
SQLite version 3.50.4 2025-07-30 19:33:53
Enter ".help" for usage hints.
sqlite> CREATE TABLE clients (
(x1...> id INTEGER PRIMARY KEY,
(x1...> username TEXT NOT NULL,
(x1...> password TEXT NOT NULL,
(x1...> fullname TEXT,
(x1...> email TEXT,
(x1...> bank_balance REAL
(x1...> );
sqlite> INSERT INTO clients (username, password, fullname, email, bank_balance) VALUES
   ...> ('Fentse@12', 'pass1', 'Fentse Smith', 'fentse.smith@gmails.com', 1200.20),
   ...> ('Tumi@2', 'pass2', 'Tumi Sithole', 'tumi.sithole@gmails.com', 3200.00),
   ...> ('Kat@3', 'pass3', 'Kat Blac', 'kat.blac@gmails.com', 780.00),
   ...> ('Bobi@15', 'pass4', 'Bobi Blanc', 'bobi.blanc@gmails.com', 5670.24),
   ...> ('Rea@7', 'pass5', 'Rea Bona', 'rea.bona@gmails.com', 2999.50);
sqlite> SELECT * FROM clients
   ...> ;
1|Fentse@12|pass1|Fentse Smith|fentse.smith@gmails.com|1200.2
2|Tumi@2|pass2|Tumi Sithole|tumi.sithole@gmails.com|3200.0
3|Kat@3|pass3|Kat Blac|kat.blac@gmails.com|780.0
4|Bobi@15|pass4|Bobi Blanc|bobi.blanc@gmails.com|5670.24
5|Rea@7|pass5|Rea Bona|rea.bona@gmails.com|2999.5
sqlite> .exit
```

The table in the image was created in a sqlite3 folder and given the name, **thedatabase.db.** This table was not used in the milestone.

To create a sqlite3 database table firstly a terminal has to be open in the folder you want to create the database.

Put in the following commands in the manner they are presented in:

```
> CREATE TABLE clients (
> id INTEGER PRIMARY KEY,
> username TEXT NOT NULL,
> password TEXT NOT NULL,
> fullname TEXT,
> email TEXT,
> bank_balance REAL
> );
> INSERT INTO clients (username, password, fullname, email, bank_balance) VALUES
  > ('Fentse@12', 'pass1', 'Fentse Smith', 'fentse.smith@gmails.com', 1200.20),
  > ('Tumi@2', 'pass2', 'Tumi Sithole', 'tumi.sithole@gmails.com', 3200.00),
  > ('Kat@3', 'pass3', 'Kat Blac', 'kat.blac@gmails.com', 780.00),
  > ('Bobi@15', 'pass4', 'Bobi Blanc', 'bobi.blanc@gmails.com', 5670.24),
  > ('Rea@7', 'pass5', 'Rea Bona', 'rea.bona@gmails.com', 2999.50);
```

ID	Username	Fullname	Email	Balance
1	Fentse@12	Fentse Smith	fentse.smith@gmails.com	1200.20
2	Tumi@2	Tumi Sithole	tumi.sithole@gmails.com	780.00
3	Kat@3	Kat Blac	kat.blac@gmails.com	3200.00
4	Bobi@15	Bobi Blanc	bobi.blanc@gmails.com	5670.24
5	Rea@7	Rea Bona	rea.bona@gmails.com	2999.50

```
PS C:\projectFolder> sqlite3 database.db
SQLite version 3.50.4 2025-07-30 19:33:53
Enter ".help" for usage hints.
sqlite> SELECT * FROM clients
   ...> ;
1|Fentse@12|pass1|Fentse Smith|fentse.smith@gmails.com|1200.2
2|Tumi@2|pass2|Tumi Sithole|tumi.sithole@gmails.com|3200.0
3|Kat@3|pass3|Kat Blac|kat.blac@gmails.com|780.0
4|Bobi@15|pass4|Bobi Blanc|bobi.blanc@gmails.com|5670.24
5|Rea@7|pass5|Rea Bona|rea.bona@gmails.com|2999.5
```

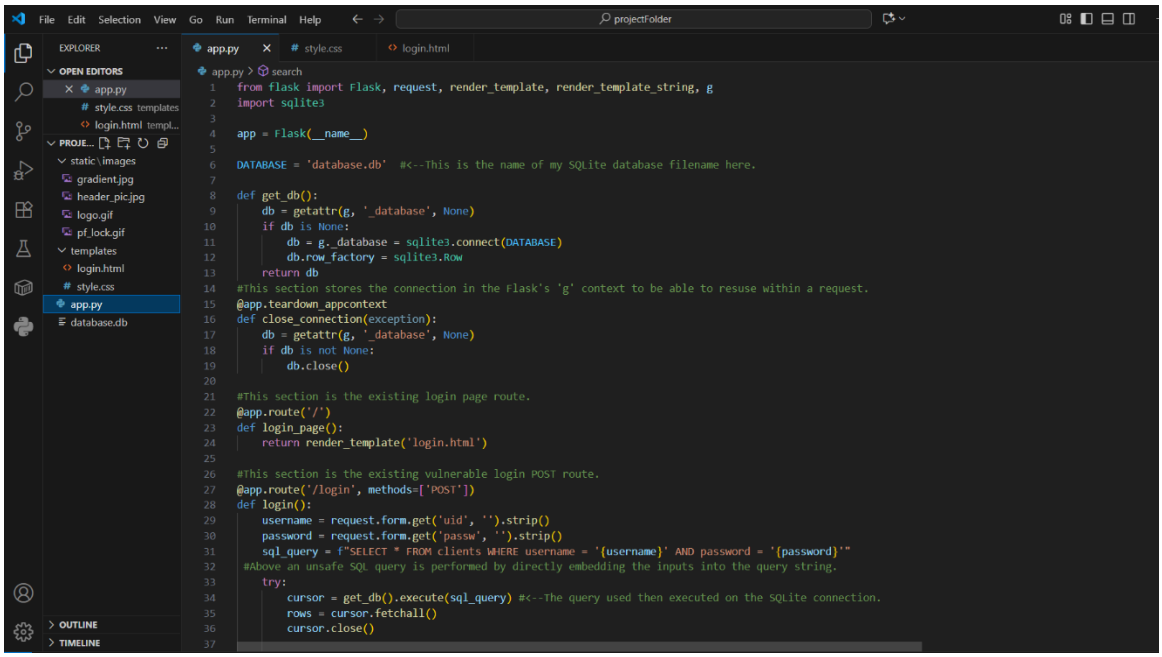
(McKenzie, 2024)

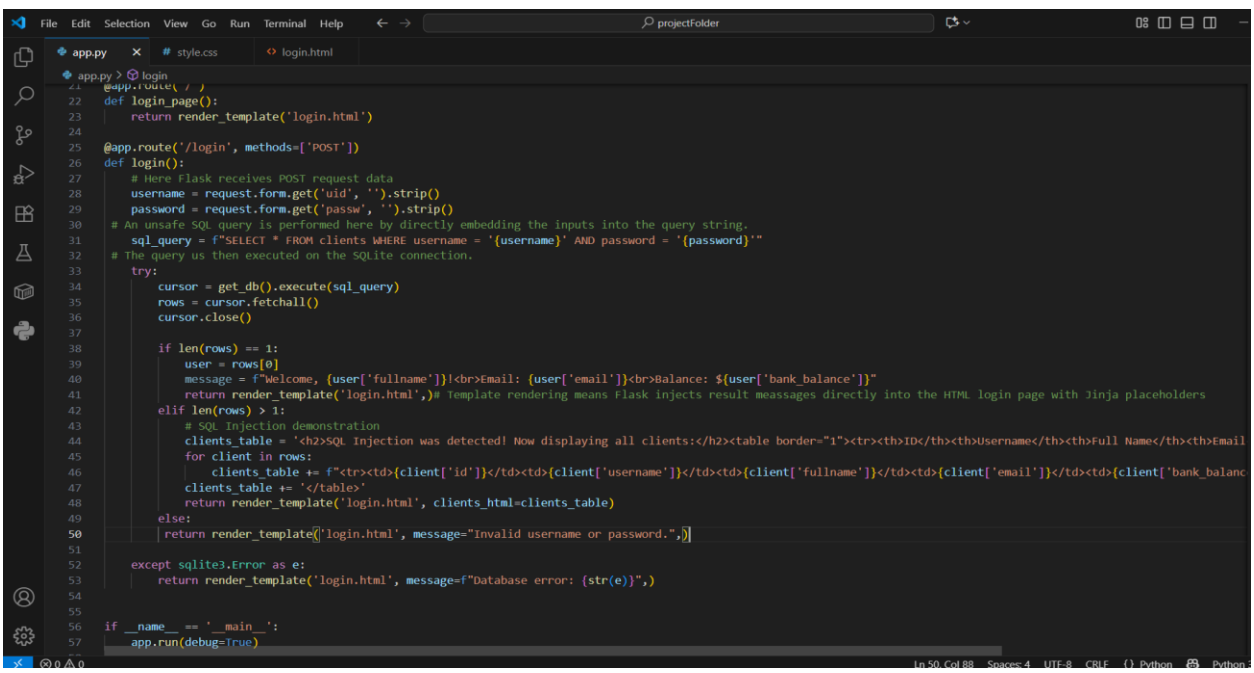
The **database.db** was the table used which was located in the folder projectFolder.

Linking webpage to database

To link the SQLite database to the cloned webpage a backend such as Python Flask was used. It acts as the middleman between the user input on the webpage and the SQLite database.

On virtual studio code in the projectFolder the file **app.py** was created and in that file the code used to link database.db to clone is pasted and shown below.





The actual code:

from flask import Flask, request, render_template, render_template_string, g

import sqlite3

app = Flask(__name__)

```
DATABASE = 'database.db'  #<--This is the name of my SQLite database filename here.

def get_db():

    db = getattr(g, '_database', None)

    if db is None:

        db = g._database = sqlite3.connect(DATABASE)

        db.row_factory = sqlite3.Row

    return db

#This section stores the connection in the Flask's 'g' context to be able to resuse within a request.

@app.teardown_appcontext

def close_connection(exception):

    db = getattr(g, '_database', None)

    if db is not None:

        db.close()

#This section is the existing login page route.

@app.route('/')

def login_page():

    return render_template('login.html')

#This section is the existing vulnerable login POST route.

@app.route('/login', methods=['POST'])

def login():

    username = request.form.get('uid', '').strip()

    password = request.form.get('passw', '').strip()

    sql_query = f"SELECT * FROM clients WHERE username = '{username}' AND password = '{password}'"

 #Above an unsafe SQL query is performed by directly embedding the inputs into the query string.

    try:

        cursor = get_db().execute(sql_query) #<--The query used then executed on the SQLite connection.

        rows = cursor.fetchall()

        cursor.close()


        if len(rows) == 1:

            user = rows[0]

            message = f"Welcome, {user['fullname']}!<br>Email: {user['email']}<br>Balance: ${user['bank_balance']}"

            return render_template('login.html', message=message)

        elif len(rows) > 1:
```

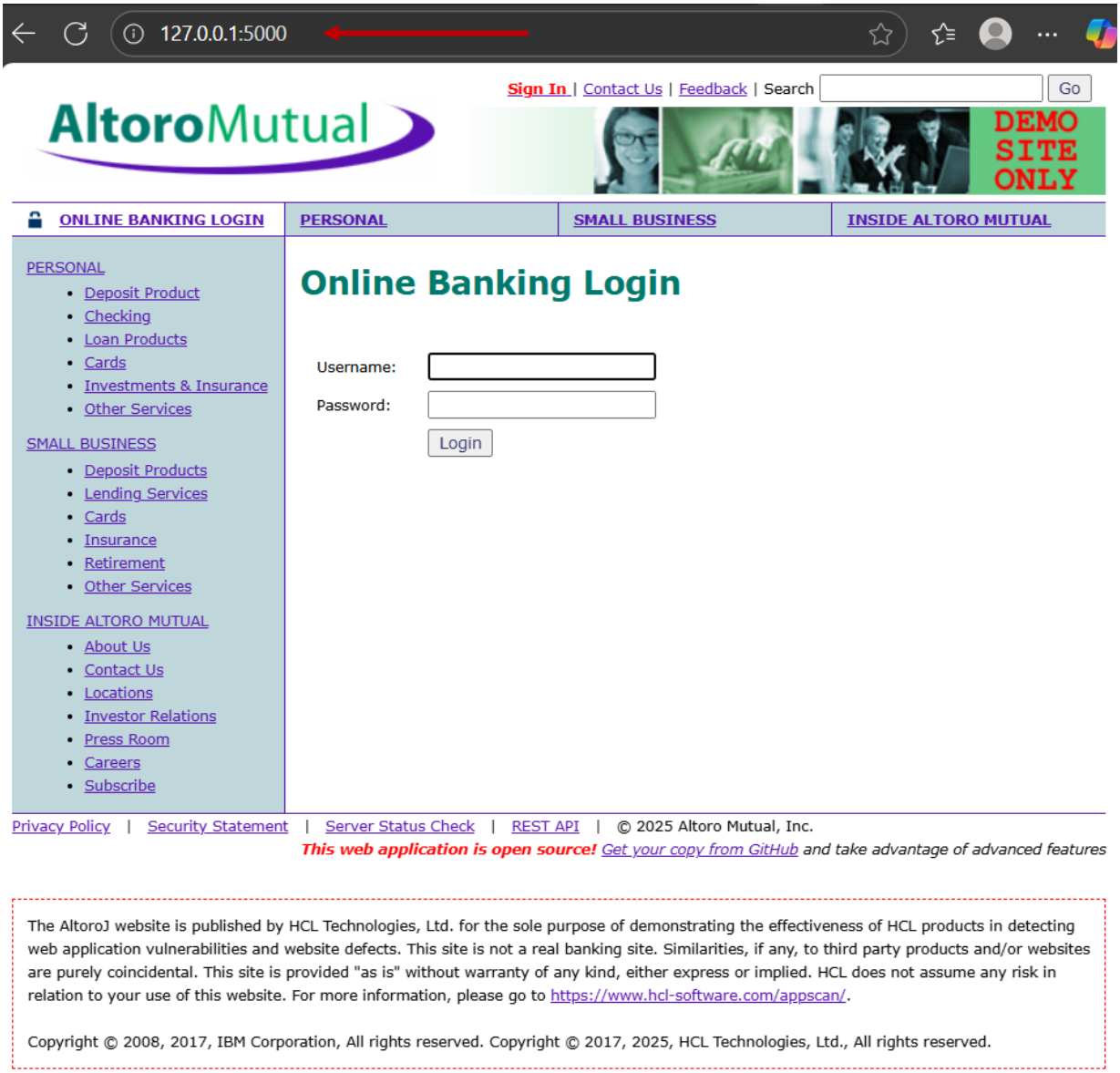
```
        clients_table = '<h2>SQL Injection was detected! Now displaying all clients:</h2><table
border="1"><tr><th>ID</th><th>Username</th><th>Full Name</th><th>Email</th><th>Balance</th></tr>'

        for client in rows:

            clients_table +=
f"<tr><td>{client['id']}</td><td>{client['username']}</td><td>{client['fullname']}</td><td>{client['email']}</td><td>{client['bank_balance'
]}</td></tr>"

        clients_table += '</table>'

        return render_template('login.html', clients_html=clients_table)

    else:

        return render_template('login.html', message="Invalid username or password.")

  except sqlite3.Error as e:

    return render_template('login.html', message=f"Database error: {str(e)}")

#This section reflectes XSS vulnerable search page.

@app.route('/search')

def search():

  user_query = request.args.get('query', '')

  # In this section Render user input unsafely with |safe to demonstrate XSS vulnerability.

  return render_template_string("""

    <html>

      <head><title>Search Page</title></head>

      <body>

        <h1>Search Results</h1>

        <p>You searched for: {{ query|safe }}</p>

      </body>

    </html>

  """, query=user_query)

if __name__ == '__main__':

  app.run(debug=True)
```

Testing the Login webpage

1. To start you need to first run the Flask application in a terminal , **python app.py.**

```
PS C:\projectFolder> python app.py
 * Serving Flask app 'app'
 * Debug mode: on
WARNING: This is a development server. Do not use it in a production deployment. Use a production WSGI server instead.
 * Running on http://127.0.0.1:5000
Press CTRL+C to quit
 * Restarting with stat
 * Debugger is active!
 * Debugger PIN: 123-368-543
127.0.0.1 - - [08/Oct/2025 09:42:31] "GET / HTTP/1.1" 200 -
127.0.0.1 - - [08/Oct/2025 09:42:31] "GET /static/images/logo.gif HTTP/1.1" 304 -
127.0.0.1 - - [08/Oct/2025 09:42:32] "GET /images/gradient.jpg HTTP/1.1" 404 -
127.0.0.1 - - [08/Oct/2025 09:42:53] "GET / HTTP/1.1" 200 -
127.0.0.1 - - [08/Oct/2025 09:42:53] "GET /static/images/logo.gif HTTP/1.1" 304 -
127.0.0.1 - - [08/Oct/2025 09:42:53] "GET /images/gradient.jpg HTTP/1.1" 404 -
127.0.0.1 - - [08/Oct/2025 09:43:44] "POST /login HTTP/1.1" 200 -
127.0.0.1 - - [08/Oct/2025 09:43:44] "GET /static/images/logo.gif HTTP/1.1" 304 -
127.0.0.1 - - [08/Oct/2025 09:43:44] "GET /images/gradient.jpg HTTP/1.1" 404 -
127.0.0.1 - - [08/Oct/2025 09:44:40] "POST /login HTTP/1.1" 200 -
127.0.0.1 - - [08/Oct/2025 09:44:40] "GET /static/images/logo.gif HTTP/1.1" 304 -
127.0.0.1 - - [08/Oct/2025 09:44:40] "GET /images/gradient.jpg HTTP/1.1" 404 -
```

2. Open the Login Webpage in a browser, **http://127.0.0.1:5000/**.



3. Login and enter password as every existing client from the database, **database.db**. When successful a welcome message, email address and bank balance will display.

127.0.0.1:5000/login

Sign In | Contact Us | Feedback | Search Go

AltoroMutual

DEMO SITE ONLY

ONLINE BANKING LOGIN | PERSONAL | SMALL BUSINESS | INSIDE ALTORO MUTUAL

PERSONAL
- Deposit Product
- Checking
- Loan Products
- Cards
- Investments & Insurance
- Other Services

SMALL BUSINESS
- Deposit Products
- Lending Services
- Cards
- Insurance
- Retirement
- Other Services

INSIDE ALTORO MUTUAL
- About Us
- Contact Us
- Locations
- Investor Relations
- Press Room
- Careers
- Subscribe

Online Banking Login

Username: Fentse@12

Password: pass1

Login

Welcome, Rea Bona!

Email: rea.bona@gmails.com

Balance: $2999.5

Privacy Policy | Security Statement | Server Status Check | REST API | © 2025 Altoro Mutual, Inc.

This web application is open source! *Get your copy from GitHub and take advantage of advanced features*

The AltoroJ website is published by HCL Technologies, Ltd. for the sole purpose of demonstrating the effectiveness of HCL products in detecting web application vulnerabilities and website defects. This site is not a real banking site. Similarities, if any, to third party products and/or websites are purely coincidental. This site is provided "as is" without warranty of any kind, either express or implied. HCL does not assume any risk in relation to your use of this website. For more information, please go to https://www.hcl-software.com/appscan/.

Copyright © 2008, 2017, IBM Corporation, All rights reserved. Copyright © 2017, 2025, HCL Technologies, Ltd., All rights reserved.

127.0.0.1:5000/login

Sign In | Contact Us | Feedback | Search Go

AltoroMutual

DEMO SITE ONLY

ONLINE BANKING LOGIN | PERSONAL | SMALL BUSINESS | INSIDE ALTORO MUTUAL

PERSONAL
- Deposit Product
- Checking
- Loan Products
- Cards
- Investments & Insurance
- Other Services

SMALL BUSINESS
- Deposit Products
- Lending Services
- Cards
- Insurance
- Retirement
- Other Services

INSIDE ALTORO MUTUAL
- About Us
- Contact Us
- Locations
- Investor Relations
- Press Room
- Careers
- Subscribe

Online Banking Login

Username: Kat@3

Password: pass3

Login

Welcome, Tumi Sithole!

Email: tumi.sithole@gmails.com

Balance: $3200.0

Privacy Policy | Security Statement | Server Status Check | REST API | © 2025 Altoro Mutual, Inc.

This web application is open source! *Get your copy from GitHub and take advantage of advanced features*

The AltoroJ website is published by HCL Technologies, Ltd. for the sole purpose of demonstrating the effectiveness of HCL products in detecting web application vulnerabilities and website defects. This site is not a real banking site. Similarities, if any, to third party products and/or websites are purely coincidental. This site is provided "as is" without warranty of any kind, either express or implied. HCL does not assume any risk in relation to your use of this website. For more information, please go to https://www.hcl-software.com/appscan/.

Copyright © 2008, 2017, IBM Corporation, All rights reserved. Copyright © 2017, 2025, HCL Technologies, Ltd., All rights reserved.

127.0.0.1:5000/login

Sign In | Contact Us | Feedback | Search Go

AltoroMutual

ONLINE BANKING LOGIN | PERSONAL | SMALL BUSINESS | INSIDE ALTORO MUTUAL

PERSONAL
- Deposit Product
- Checking
- Loan Products
- Cards
- Investments & Insurance
- Other Services

SMALL BUSINESS
- Deposit Products
- Lending Services
- Cards
- Insurance
- Retirement
- Other Services

INSIDE ALTORO MUTUAL
- About Us
- Contact Us
- Locations
- Investor Relations
- Press Room
- Careers
- Subscribe

Online Banking Login

Username: Bobi@15

Password: pass4

Login

Welcome, Kat Blac!

Email: kat.blac@gmails.com

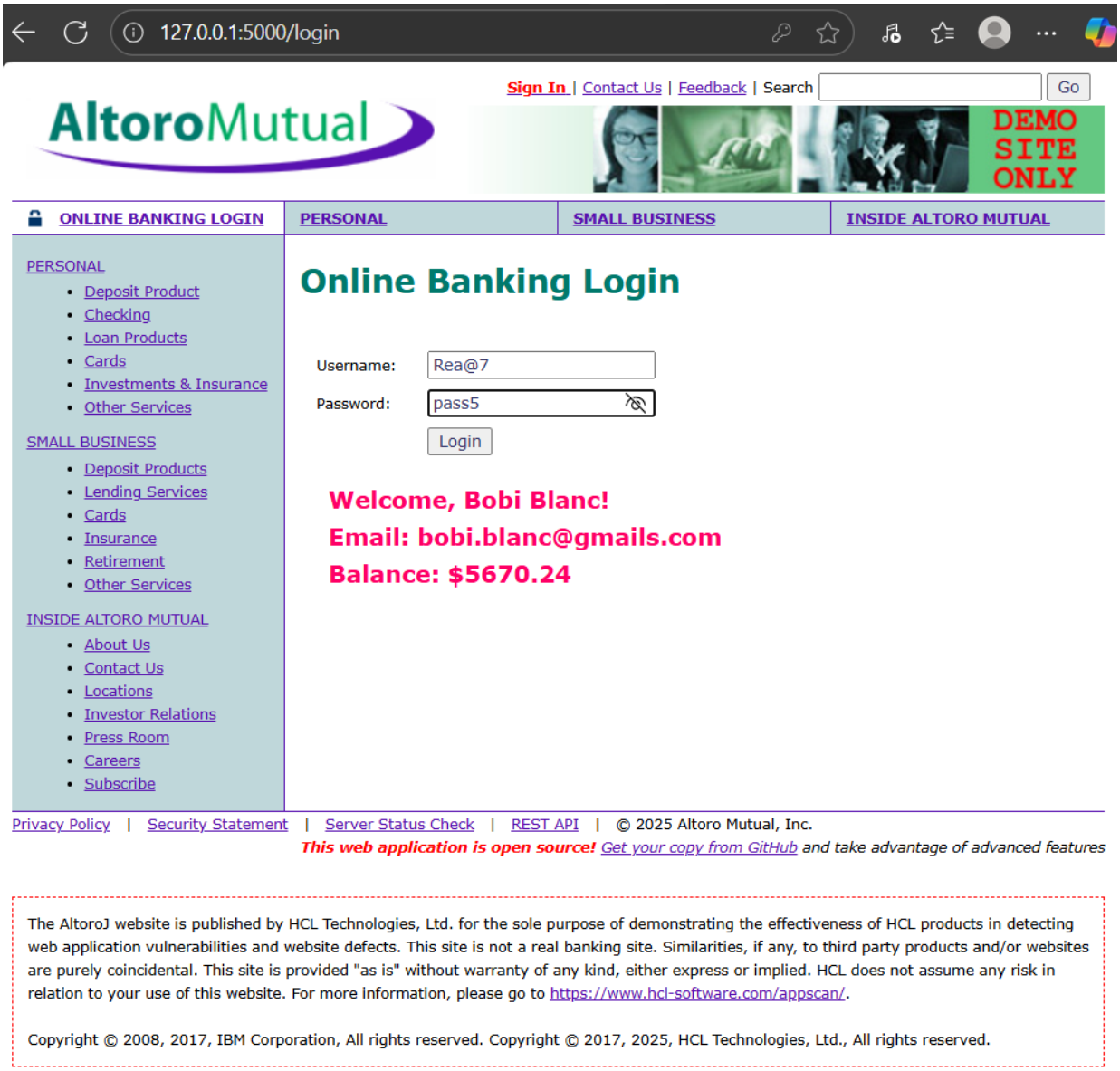
Balance: $780.0

Privacy Policy | Security Statement | Server Status Check | REST API | © 2025 Altoro Mutual, Inc.

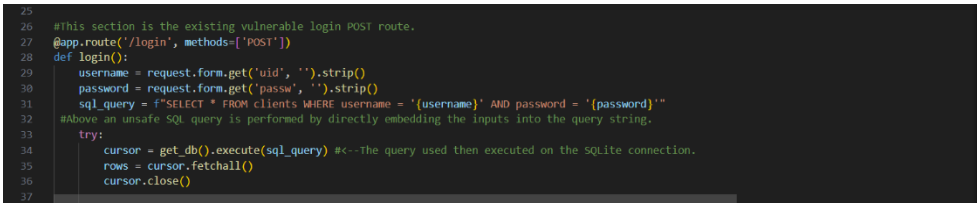
This web application is open source! *Get your copy from GitHub and take advantage of advanced features*

The AltoroJ website is published by HCL Technologies, Ltd. for the sole purpose of demonstrating the effectiveness of HCL products in detecting web application vulnerabilities and website defects. This site is not a real banking site. Similarities, if any, to third party products and/or websites are purely coincidental. This site is provided "as is" without warranty of any kind, either express or implied. HCL does not assume any risk in relation to your use of this website. For more information, please go to https://www.hcl-software.com/appscan/.

Copyright © 2008, 2017, IBM Corporation, All rights reserved. Copyright © 2017, 2025, HCL Technologies, Ltd., All rights reserved.



SQL injection

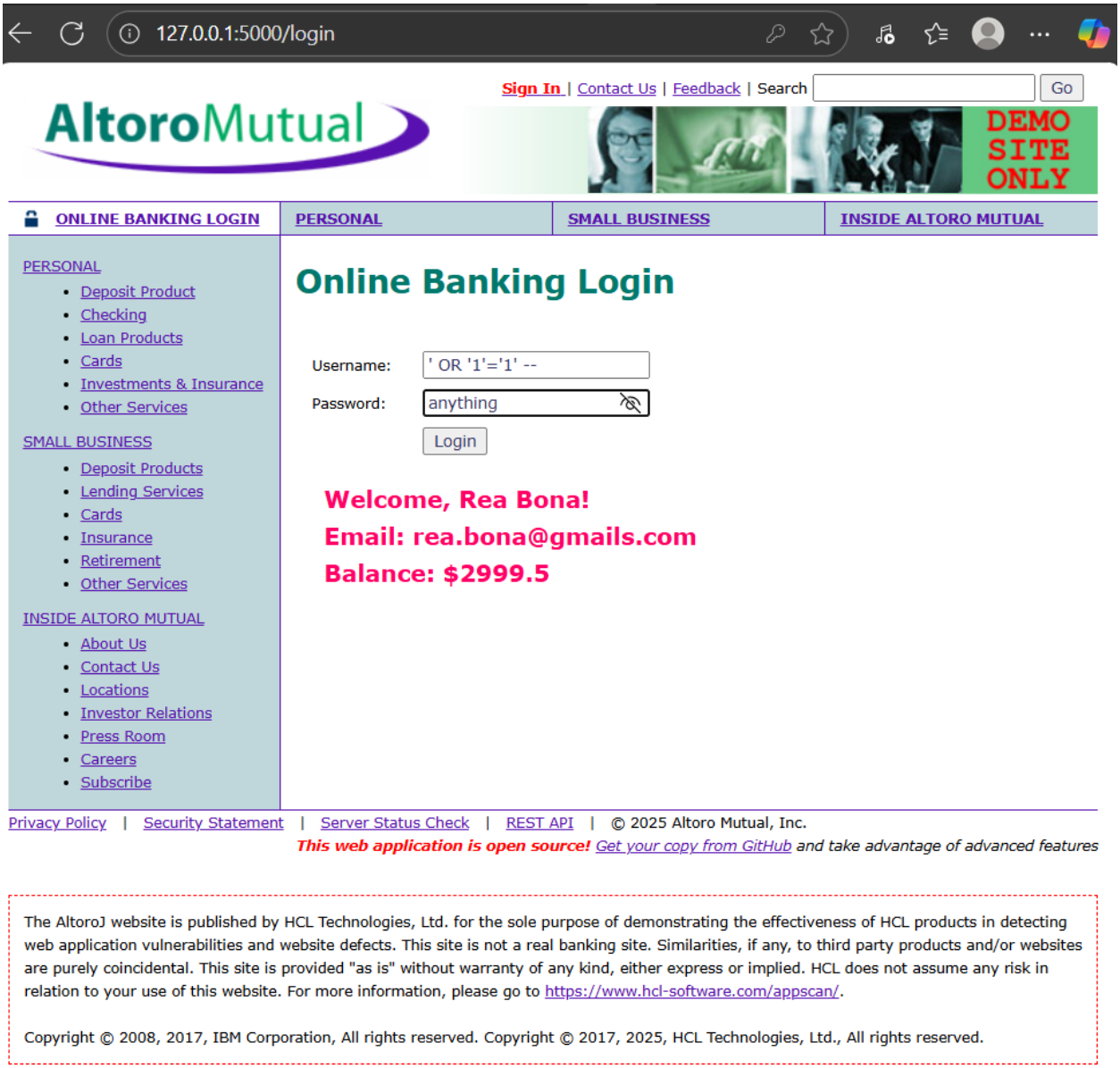


```
#This section is the existing vulnerable login POST route.
@app.route('/login', methods=['POST'])
def login():
    username = request.form.get('uid', '').strip()
    password = request.form.get('passw', '').strip()
    sql_query = f"SELECT * FROM clients WHERE username = '{username}' AND password = '{password}'"
#Above an unsafe SQL query is performed by directly embedding the inputs into the query string.
    try:
        cursor = get_db().execute(sql_query) #<--The query used then executed on the SQLite connection.
        rows = cursor.fetchall()
        cursor.close()
```

This is the unsecure SQL query code:

sql_query = f"SELECT * FROM clients WHERE username = '{username}' AND password = '{password}'"

It is unsecure because string concatenation mixes code and data (NetworkChuck, 2022).

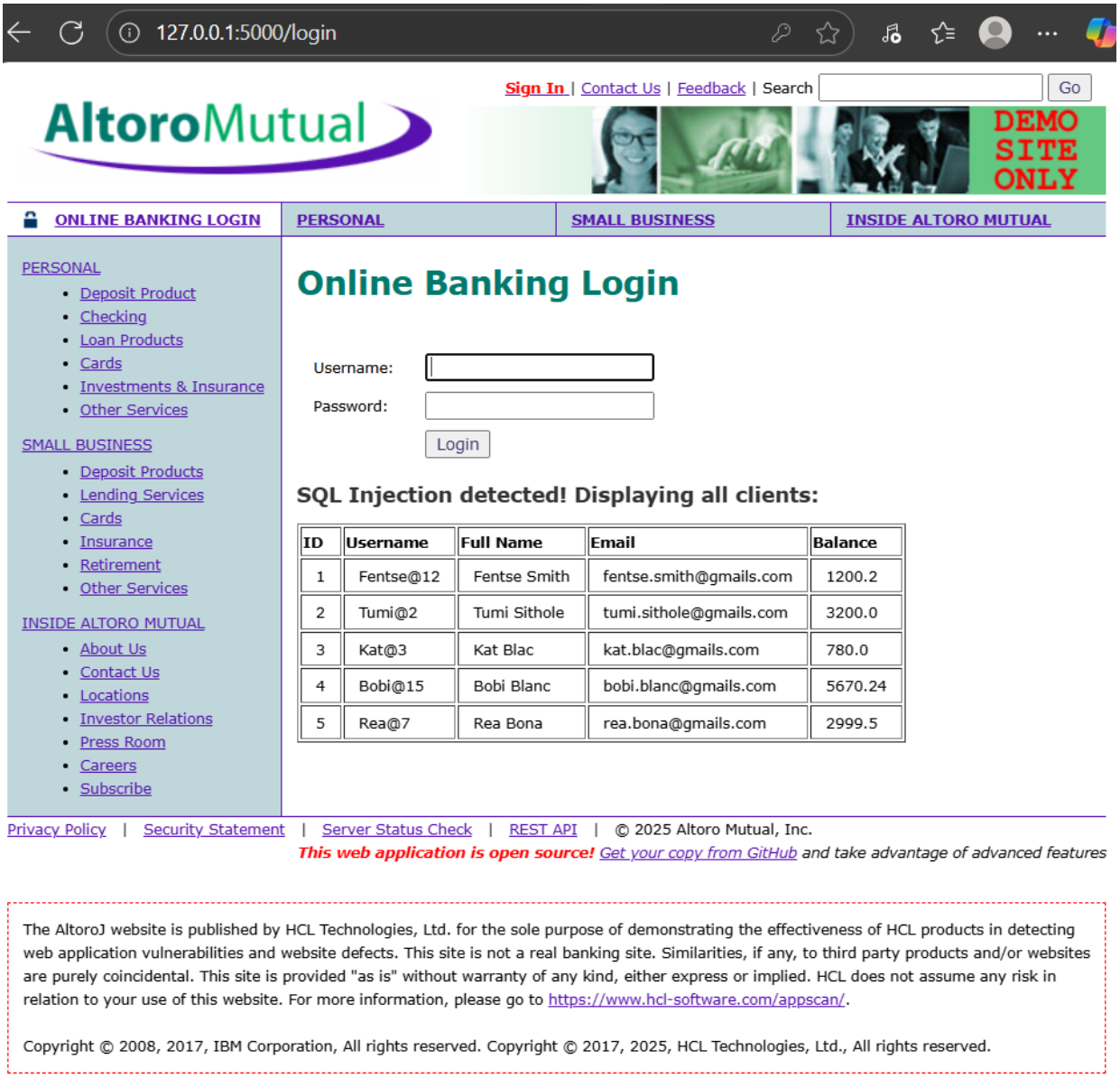


This payload, **‘ OR ‘1’=’1’ --** , is used as the username. It is then interpreted as ,

SELECT * FROM clients WHERE username = '' OR '1'='1' -- ' AND password = '', which causes a display of all clients resulting in a successful injection (NetworkChuck, 2022).

--: this means comment, so anything that comes after the comment is seen as a comment and no longer a query**.** Hence the injection becomes a success.

‘1’=’1’: this is a condition that is always true, so the query returns all the rows.



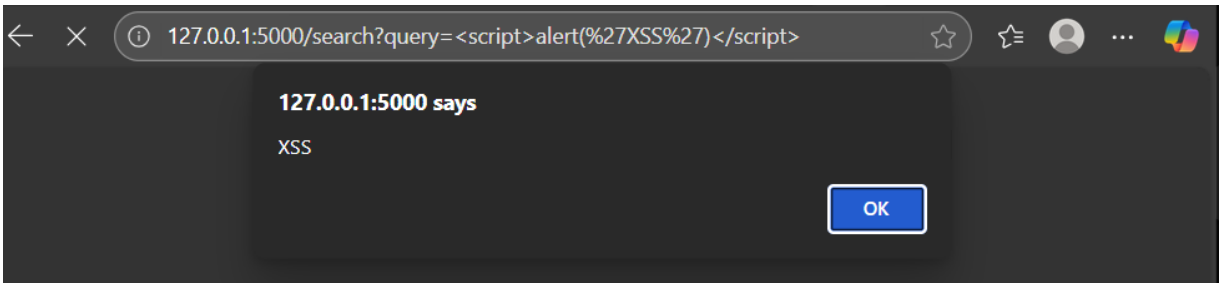
The display of the clients in the database and a message about SQL Injection detection.

Cross- Site Scripting

```python
#This section reflectes XSS vulnerable search page.
@app.route('/search')
def search():
    user_query = request.args.get('query', '')
    # In this section Render user input unsafely with |safe to demonstrate XSS vulnerability.
    return render_template_string("""
        <html>
            <head><title>Search Page</title></head>
            <body>
                <h1>Search Results</h1>
                <p>You searched for: {{ query|safe }}</p>
            </body>
        </html>
    """, query=user_query)


if __name__ == '__main__':
    app.run(debug=True)
```

This reflects code XSS from **app.py** also known as Python Flask :

```
@app.route('/search')

def search():

  user_query = request.args.get('query', '')

  # In this section Render user input unsafely with |safe to demonstrate XSS vulnerability.

  return render_template_string("""

    <html>

      <head><title>Search Page</title></head>

      <body>

        <h1>Search Results</h1>

        <p>You searched for: {{ query|safe }}</p>

      </body>

    </html>

  """, query=user_query)
```



To execute XSS put , http://127.0.0.1:5000/search?query=<script>alert('XSS')</script>, in a browser and the alert with “XSS” will show, indicating your injection was successful (Jim Manico, n.d.).

127.0.0.1: this is a network address, loopback Ip address

5000: this is a port number where the Flask web server is listening for HTTP requests

<script>: this is a JavaScript code with HTML script tags (Jim Manico, n.d.)

alert('XSS')</script>: this is a simple JavaScript command that shows a pop-up alert showing text “XSS” (Jim Manico, n.d.)

References

AmanBytes, 2024. *Learn PHP Language In 10 Minutes!! PHP Language Tutorial*. [Online]
Available at: https://www.youtube.com/watch?v=2s6mIboARCM
[Accessed 7 october 2025].

geeksforgeeks, 2025. *SQL Injection*. [Online]
Available at: https://www.geeksforgeeks.org/sql/sql-injection/
[Accessed 8 october 2025].

Jim Manico, J. W. D. W. A. W. R. A. J. A. S. J. K. I. E. Y. k. V. K. G. O., n.d. *Cross Site Scripting (XSS)*. [Online]
Available at: https://owasp.org/www-community/attacks/xss/
[Accessed 8 october 2025].

McKenzie, C., 2024. *Introduction to Sqlite for Beginners: Full Sqlite3 DB Tutorial*. [Online]
Available at: https://www.youtube.com/watch?v=BNUvVDbQ0J0&t=15s
[Accessed 7 october 2025].

NetworkChuck, 2022. *SQL Injections are scary!! (hacking tutorial for beginners)*. [Online]
Available at: https://www.youtube.com/watch?v=2OPVViV-GQk
[Accessed 7 october 2025].

OHkae, 2021. *How to copy the HTML, CSS and JS code from any webpage on Google!*. [Online]
Available at: https://www.youtube.com/watch?v=9PVh4XOq348
[Accessed 7 october 2025].